RMSProp & GDM Merge

$$w_{t+1} = w_t - \frac{\alpha}{\sqrt{V_t^c + \varepsilon}} u_t^c$$

$$u_t = \beta_2 u_{t-1} + (1-\beta_2) \nabla w_t$$

$$V_t = \beta_1 V_{t-1} + (1-\beta_1) (\nabla w_t)^2$$

$$V_t^c = \frac{V_t}{1-\beta_1^t} \qquad u_t^c = \frac{u_t}{1-\beta_2^t}$$

HW:- ADAMW

Unsupervised Learning : K-Means Algo.

No. of channels in input & filter → remain same

Many to many → encoder to decoder

BPTT (BackPropagation Through Time)

$$\hat{c}^{<t>} = \tanh(\text{linear}(\Gamma_r c^{<t-1>}, x^{<t>}))$$

$$\Gamma_u = \sigma(\text{linear}(c^{<t-1>}, x^{<t>}))$$

$$\Gamma_r = \sigma(\text{linear}(c^{<t-1>}, x^{<t>}))$$

$$c^{<t>} = \Gamma_u \hat{c}^{<t>} + (1-\Gamma_u) c^{<t-1>}$$

$$h_t = a^{<t>} = c^{<t>}$$

LSTM :-

$$\hat{c}^{<t>} = \tanh(\text{lin}(a^{<t-1>}, x^{<t>}))$$

$$\Gamma_u = \sigma(\text{lin}(a^{<t-1>}, x^{<t>}))$$

$$\Gamma_f = \sigma(\text{lin}(a^{<t-1>}, x^{<t>}))$$

$$\Gamma_o = \sigma(\text{lin}(a^{<t-1>}, x^{<t>}))$$

$$c^{<t>} = \Gamma_u \hat{c}^{<t>} + \Gamma_f c^{<t-1>}$$

$$h^{<t>} = \Gamma_o \tanh c^{<t>}$$

GRU :-

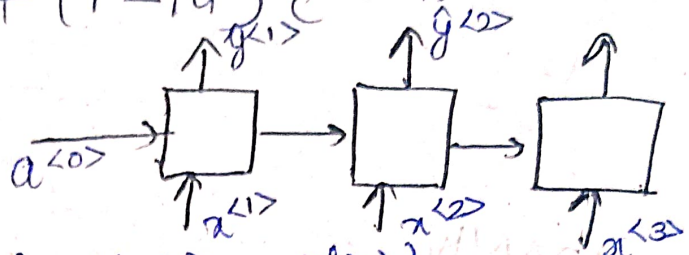
$$\tilde{c}^{<t>} = \tanh(\text{lin}(\Gamma_r * c^{<t-1>}, x^{<t>}))$$

$$\Gamma_u = \sigma(\text{lin}(c^{<t-1>}, x^{<t>}))$$

$$\Gamma_r = \sigma(\text{lin}(c^{<t-1>}, x^{<t>}))$$

$$c^{<t>} = \Gamma_u * \tilde{c}^{<t>} + (1 - \Gamma_u) c^{<t-1>}$$

$$a^{<t>} = c^{<t>}$$



LSTM :-

$$\tilde{c}^{<t>} = \tanh(\text{lin}(a^{<t-1>}, x^{<t>}))$$

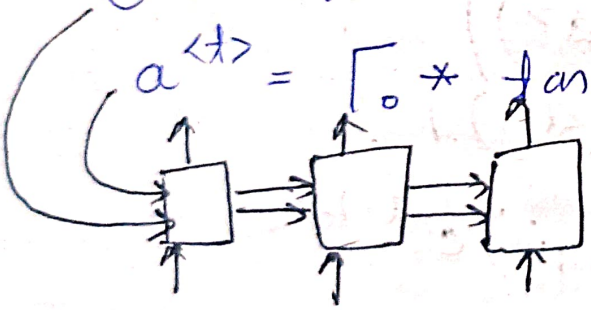
$$\Gamma_u = \sigma(\text{lin}(a^{<t-1>}, x^{<t>}))$$

$$\Gamma_f = \sigma(\text{lin}(a^{<t-1>}, x^{<t>}))$$

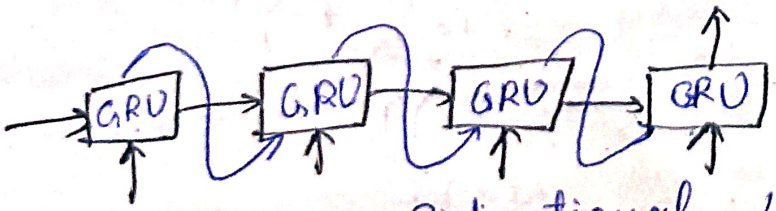
$$\Gamma_o = \sigma(\text{lin}(a^{<t-1>}, x^{<t>}))$$

$$c^{<t>} = \Gamma_u * \tilde{c}^{<t>} + \Gamma_f * c^{<t-1>}$$

$$a^{<t>} = \Gamma_o * \tanh c^{<t>}$$



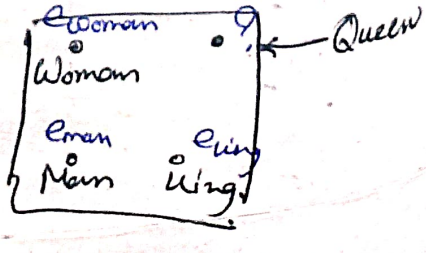
Sentiment Analysis < Pos (1) / Neg (0)



→ Many to One architecture

BiLSTM → Bidirectional LSTM

$$a^{<t>} = [\overrightarrow{a}^{<t>}, \overleftarrow{a}^{<t>}]$$

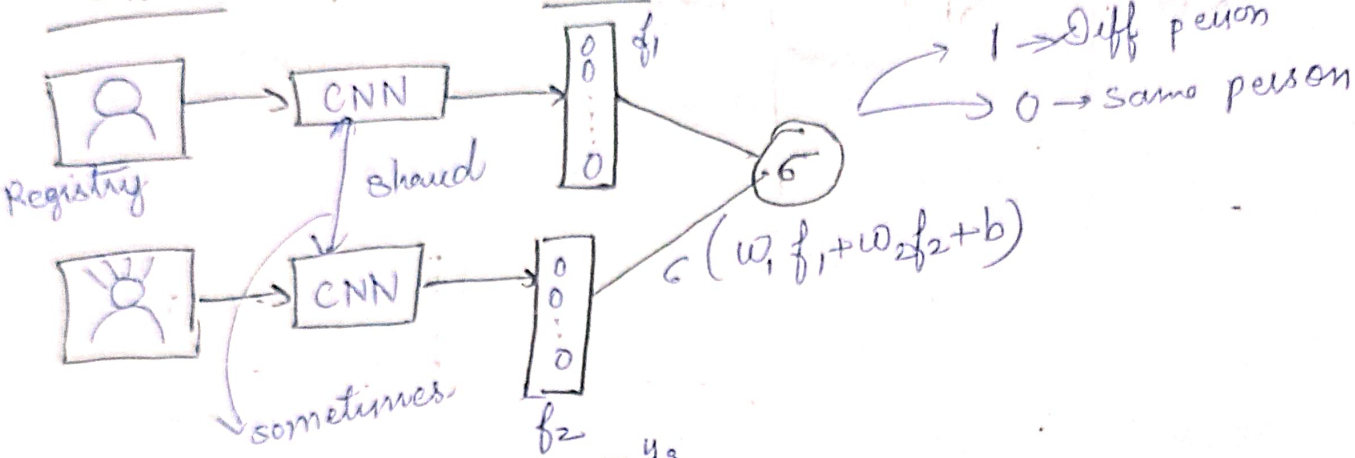


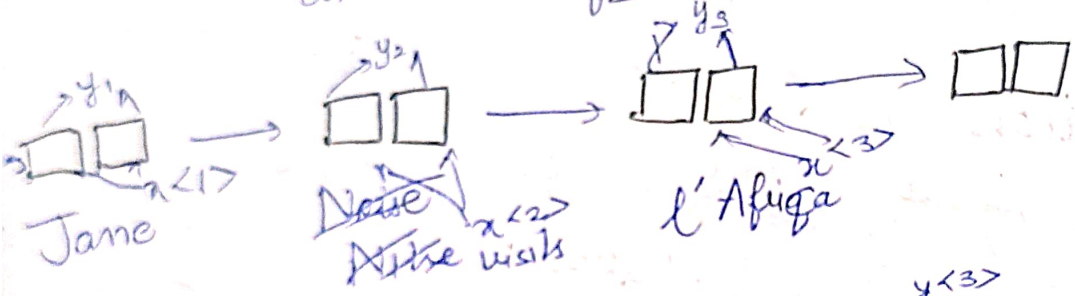
$$sim(e_w, (e_{king} - e_{man} + e_{woman}))$$

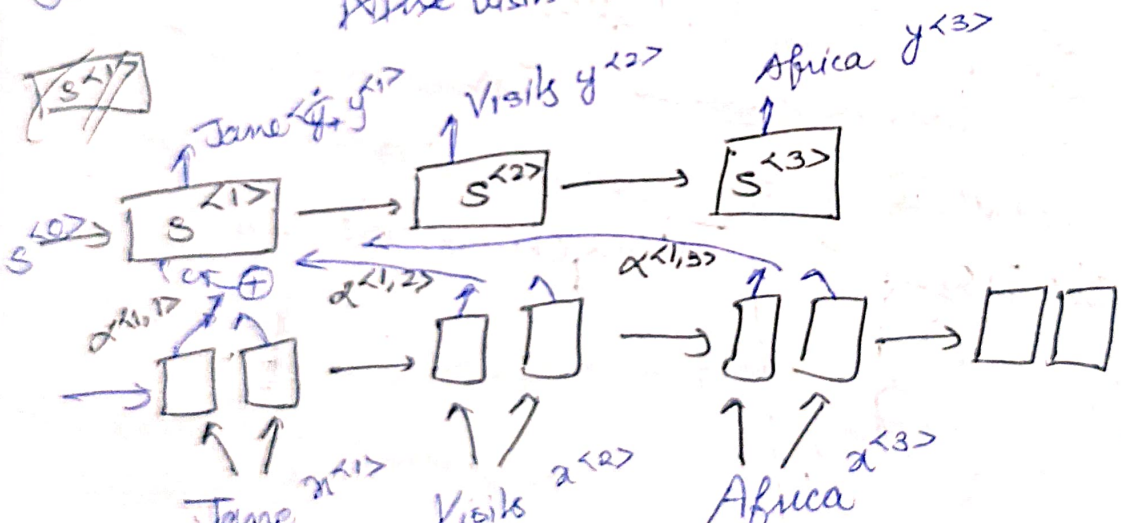
cosine similarity (u, v) = $\frac{u^T v}{\|u\|_2 \|v\|_2}$

(sim)

Siamese Architecture







$$a^{<1>} = [\vec{a}^{<1>}, \overleftarrow{a}^{<1>}]$$

$$a^{<t'>} = [\vec{a}^{<t'>}, \overleftarrow{a}^{<t'>}]$$

α → attention weights

$\alpha^{<t,t'>}$ = amount of attention $y^{<t>}$ pay to $a^{<t'>}$

$$\alpha^{<t,t'>} = \frac{\exp(e^{<t,t'>})}{\sum_{i=1}^{T} \exp(e^{<t,t'>})}$$

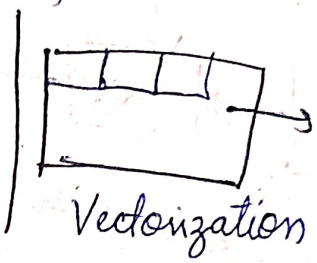
fatt → 1 layer simple neural network

Attention (2014)

Attention is all you need (2017)

RNN → facing issue ← long sequence, vanishing gradient problem

RNN | GRU (Solved) | LSTM (V.G. to some extent) | (SA) Self Attention, MHA (Multihead attention)



Self Attention $A^{<1>}, A^{<2>}, \ldots A^{<5>}$

Jane visit Africa in September → self attention

MHA: for loop (parallelization over SA)

$A(q, k, v)$ = attention based vector representation

↑ calculate for each word → $A^{<1>}, \ldots A^{<5>}$

RNN attention

$$\alpha(t, t') = \frac{\exp(e^{<t,t'>})}{\sum_{t=1}^{T_x} \exp(e^{<t,q'>})}$$

Transformer Attention

$$A(q, k, V) = \sum_i \frac{\exp(q, k^{<i>})}{\sum_j \exp(q \cdot k^{<j>})}$$

Key	Value
Brand	
Color	{Red, Green, Black}

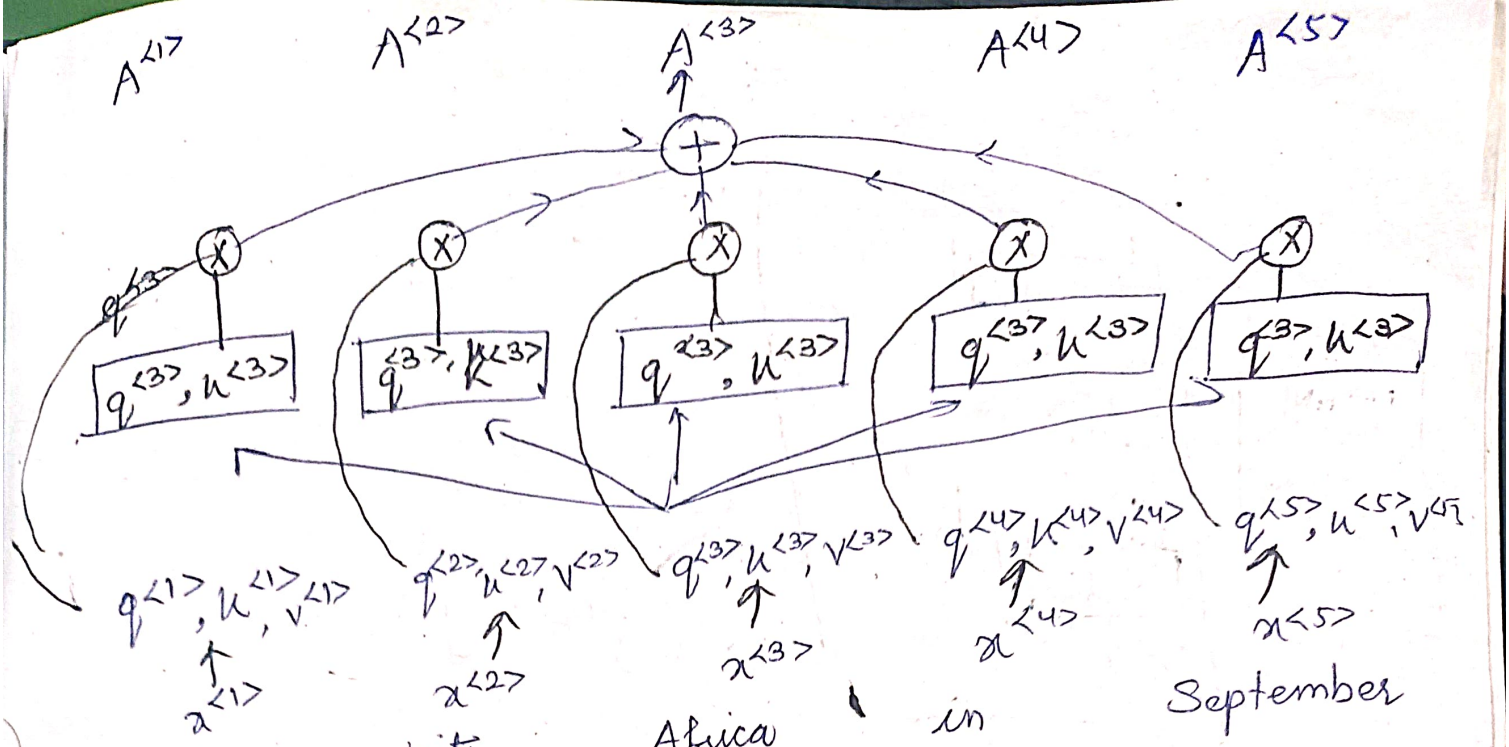
→ Query:- Which brand's color?

Q	K	V
Query	Key	Value
$q^{<1>}$	$k^{<1>}$	$V^{<1>}$
$q^{<2>}$	$k^{<2>}$	$V^{<2>}$
$q^{<3>}$	$K^{<3>}$	$V^{<3>}$

$q^{<1>}, k^{<1>}, v^{<1>}$ $q^{<2>}, k^{<2>}, v^{<2>}$ $q^{<3>}, k^{<3>}, v^{<3>}$ $q^{<4>}, k^{<4>}, v^{<4>}$ $q^{<5>}, k^{<5>}, v^{<5>}$

$x^{<1>}$ Jane $x^{<2>}$ visits $x^{<3>}$ Africa $x^{<4>}$ in $x^{<5>}$ September

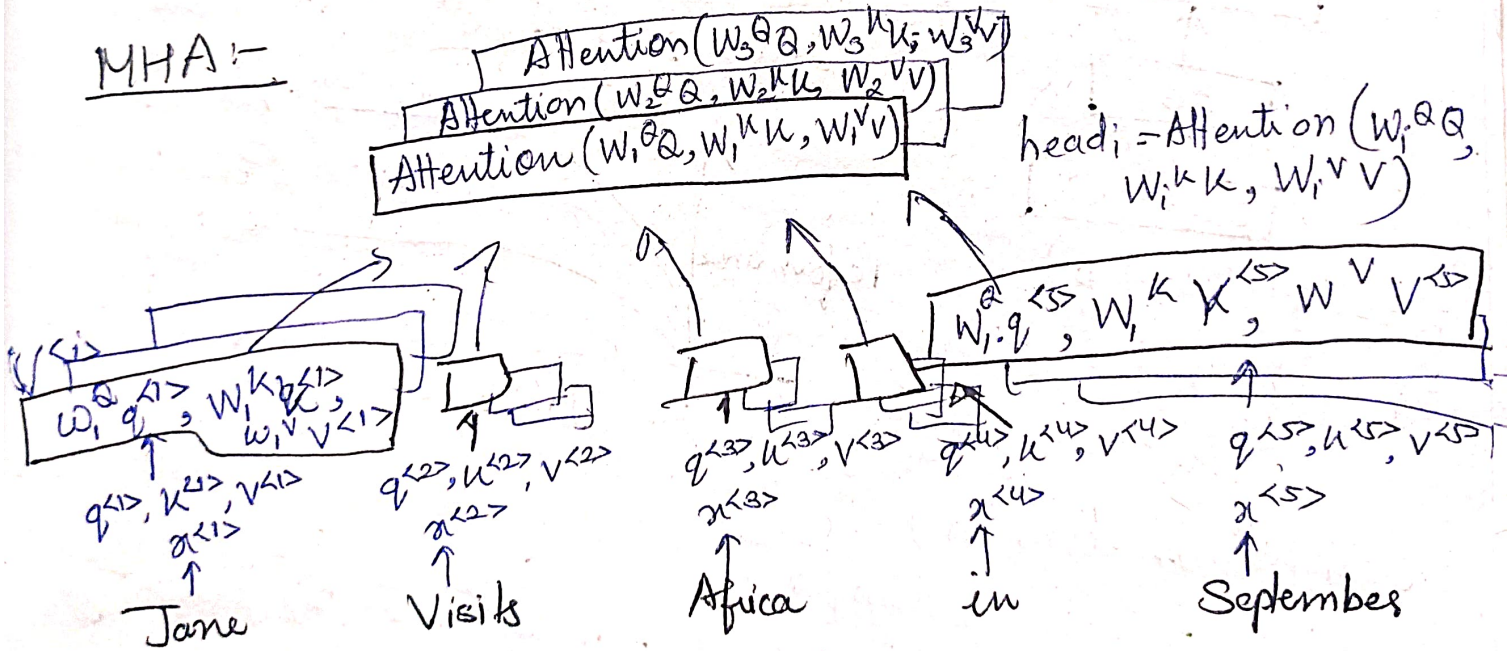
$q^{<1>}, q^{<2>}, q^{<3>}, q^{<4>}, q^{<5>}$



$$\text{Attention}(Q,K,V) = \text{Softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

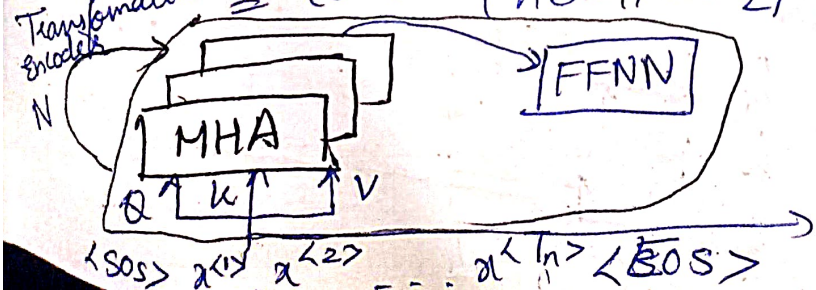
$$A(q,k,v) = \sum_i \frac{\exp(q \cdot k^{<i>})}{\sum_j \exp(q \cdot k^{<j>})} v^{<i>}$$

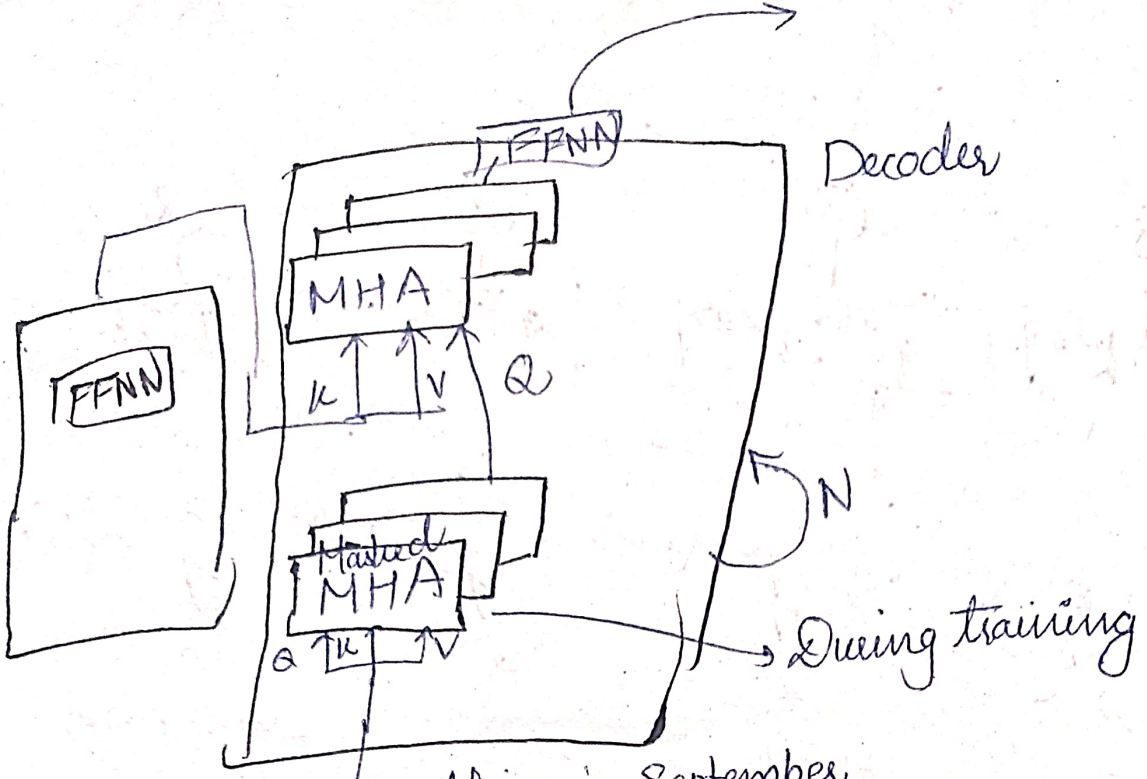
MHA :-



W_1^Q, W_1^K, W_1^V → what is happening ?

$$MHA(Q,K,V) = \text{concat}(head_1, head_2, \ldots head_h)$$



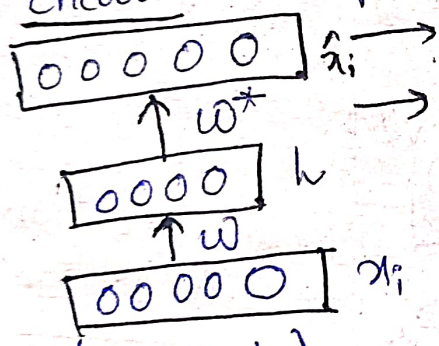


* Auto Encoder → Special type of FFNN

① Encodes the input x_i into hidden representation

② Decodes the i/p again from h

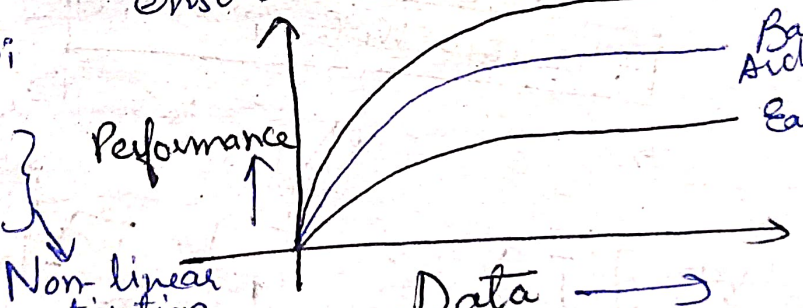
③ AE is trained to minimize a central loss func^n which will ensure that $\hat{x}_i$ is close to x_i



$h = g(Wx_i + b)$

$\hat{x}_i = f(W^* h + c)$

Non-linear activation func^n



$dim(\hat{x}_i) = dim(h)$

Let $dim(h) < dim(x_i)$

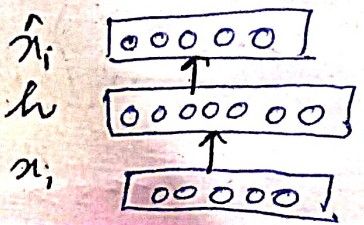
if we are still able to reconstruct $\hat{x}_i$ perfectly from h, then what does it say about h?

h is a loss free encoding of x_i

It captures all the important characteristics of x_i

Under complete AE

Over Complete AE



Do you see an analogy with PCA?

- choice of $f(.)$ and $g(.)$
- choice of loss func^n

0 1 1 0 1 (binary input)

Let all i/p (s) are binary each $x_{ij} \in \{0,1\}$

$$X = \begin{bmatrix} x_{11} & x_{12} & \dots & x_{1n} \\ \vdots & & & \\ x_{m1} & x_{m2} & \dots & x_{mn} \end{bmatrix} \leftarrow x_i$$

— Which of the following func^n should be most appropriate for the DECODER ?

(A) $\hat{x}_i = \tanh(W^* h + c)$

(B) $\hat{x}_i = W^* h + c$

(C) $\hat{x}_i = \sigma(W^* h + c)$ ✓

$f \rightarrow \sigma$ (Need values b/w 0 & 1)

Tranforma)

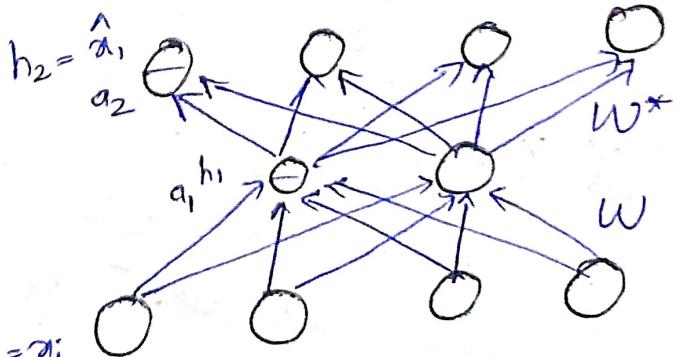
* The objective of AE to reconstruct $\hat{x}_i$ to be as close to x_i as possible.

$$\sum_{i=1}^{m}\sum_{j=1}^{n}(\hat{x}_{ij} - x_{ij})^2/m \approx \min_{W,W^*,c,b} \frac{1}{m}\sum_{i=1}^{m}(\hat{x}_i - x_i)^T(\hat{x}_i - x_i)$$

↳ Min W, W^*, c, b

$$\approx \min_{W,W^*,c,b} (\hat{X} - X)^T(\hat{X} - X)$$

We can train AE just like a FFNN with Backpropagation



$$\frac{\partial L}{\partial W^*} = \frac{\partial L}{\partial h_2}\frac{\partial h_2}{\partial a_2}\frac{\partial a_2}{\partial W^*}$$

$$\frac{\partial L}{\partial W} = \frac{\partial L}{\partial h_2}\frac{\partial h_2}{\partial a_2}\frac{\partial a_2}{\partial h_1}\frac{\partial h_1}{\partial a_1}\frac{\partial a_1}{\partial W}$$

$$L(\theta) = (\hat{x}_i - x_i)^T(\hat{x}_i - x_i)$$

$$\frac{\partial L}{\partial h_2} = \frac{\partial L}{\partial \hat{x}_i} = \nabla_{\hat{x}_i} \{(\hat{x}_i - x_i)^T (\hat{x}_i - x_i)\}$$
$$= 2(\hat{x}_i - x_i)$$

For a single n-dimensional i^{th} i/p, we can use the following loss func^n (Binary i/p)

$$\min \left\{ -\sum_{i=1}^{n} \sum_{j=1}^{n} x_{ij} \log \hat{x}_{ij} + (1 - x_{ij}) \log (1 - \hat{x}_{ij}) \right\}$$

Initialized

0 0	0
0 1	log 0
1 0	log 0
1 1	0

$$\frac{\partial L}{\partial h_2} = \frac{-x_{ij}}{\hat{x}_{ij}} + \frac{1 - x_{ij}}{1 - \hat{x}_{ij}}$$

LINK B/W PCA & AE

We will now see that the encoder part of AE is equivalent to PCA, if we

→ use a linear encoder
→ use a linear decoder
→ use squared error loss f
→ Normalize the i/ps to

$$\hat{x}_{ij} = \frac{1}{\sqrt{m}} \left(x_{ij} - \underbrace{\frac{1}{m} \sum_{k=1}^{m} x_{kj}}_{\text{Zero mean along each line } j} \right)$$

$$X^T X = \frac{1}{m} (X')^T X' \qquad X = \frac{1}{m} X'$$

is cov. matrix (recall from PCA)

We will show that if we use linear decoder & square error loss then the optimal sol^n of objective func^n $\frac{1}{m} \sum_{i=1}^{m} \sum_{j=1}^{n} (x_{ij} - \hat{x}_{ij})^2$ is obtained when we use linear encoder

$$\min_{\theta} \sum_{i=1}^{m} \sum_{j=1}^{n} (x_{ij} - \hat{x}_{ij})^2$$ Forbeas Norm

This is equivalent to

$$\min_{W^*H} \left(\|X - HW^*\|_F \right)^2$$

$$\|A\|_F = \sqrt{\sum_{i=1}^{m} \sum_{j=1}^{n} a_{ij}^2}$$

From SVD, we know that optimal soln to the above problem

$$HW^* = U_{\cdot,\leq k} \Sigma_{k,k} V^T_{\cdot,\leq k}$$

We will show H is a linear encoding and find an expression for the encoder weights W

$$H = U_{\cdot,\leq k} \Sigma_{k,k}$$

$$= (XX^T)(XX^T)^{-1} U_{\cdot,\leq k} \Sigma_{k,k}$$

$$= (XV\Sigma^T U^T)(U\Sigma\Sigma^T U^T)^{-1} U_{\cdot,\leq k} \Sigma_{k,k}$$

$$= (XV\Sigma^T U^T)(U\Sigma \underbrace{V^T V}_{I} \Sigma^T U^T)^{-1} U_{\cdot,\leq k} \Sigma_{k,k}$$

$$= XV\Sigma^T U^T (U\Sigma\Sigma^T U^T)^{-1} U_{\cdot,\leq k} \Sigma_{k,k}$$

$$= XV\Sigma^T U^T U (\Sigma\Sigma^T)^{-1} U^T U_{\cdot,\leq k} \Sigma_{k,k}$$

$$= XV\Sigma^T (\Sigma\Sigma^T)^{-1} U^T U_{\cdot,\leq k} \Sigma_{k,k}$$

$$= XV\Sigma^T \Sigma^{T-1} \Sigma^{-1} U^T U_{\leq k} \Sigma_{k,k}$$

$$= XV\Sigma^{-1} I_{\leq k} \Sigma_{k,k} = XV I_{\leq k}$$

$$H = XV_{\leq k}$$

H is a linear transformation of X and $W = V_{\cdot,\leq k}$

We have encoder $W = V_{\cdot,\leq k}$

From SVD, we know V is the matrix of eigen vectors of X^TX from PCA, we know P is the matrix of eigen vectors of cov. matrix

We saw earlier, $\hat{X}_{ij} = \frac{1}{\sqrt{m}} \left(x_{ij} - \frac{1}{m} \sum_{k=1}^{m} x_{kj} \right)$

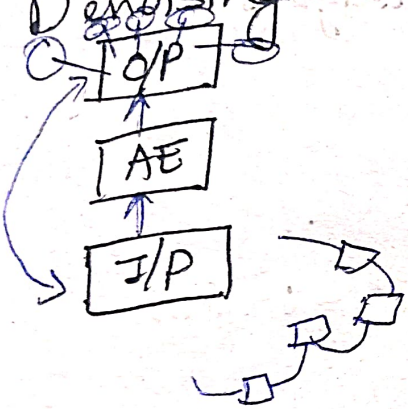
Then $X^T X$ is the cov. matrix

⇒ The encoder matrix for linear auto encoder (W) and the projection (P) for PCA could be same.

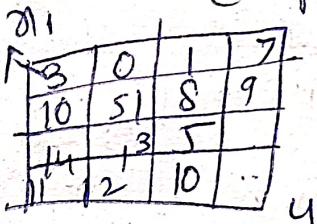
Regularization in AE

$$\min_{\theta, W, W^*, b, c} \frac{1}{m} \sum_{i=1}^{m} \sum_{j=1}^{n} (\hat{x}_{ij} - x_{ij})^2 + \lambda \|\theta\|^2$$

Denoising AE (Read by yourself)



Computer Vision

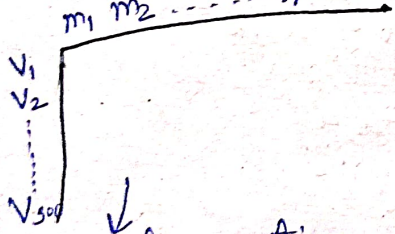


4x4

Patchwise → RNN

* Take image as sequence of patches & process

Sparse AE



Majority portion → zero